

Moteur d'exécution

Sommaire

Présentation du cours

[Projet](#)

[Moteur d'exécution](#)

[Activations des fonctions](#)

[Adresses](#)

[Table des symboles](#)

```
position = initial + vitesse*60
```

analyseur lexical

```
id1 := id2 + id3*60
```

analyseur syntaxique

```

      =
    /  \
  id1   +
   /  \
 id2   *
   /  \
 id3   60
  
```

analyseur sémantique

```

    /  \
  id1   +
   /  \
 id2   *
   /  \
 id3  inttoreal
      |
      60
  
```

Table des symboles

1	position	...
2	initial	...
3	vitesse	...
4		
5		
...		

Partie dépendante
de l'architecture cible

```

MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
  
```

générateur de code cible

```

temp1 := id3 * 60.0
id1 := id2 + temp1
  
```

optimiseur de code

```

temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
  
```

générateur de code intermédiaire

Objectifs du cours

> gcc toto.c

toto.c:1: erreur d'analyse syntaxique before ',' token

/tmp/ccImHOOB.o(.text+0x27): In function 'main' : undefined reference to 'get'

toto.c: Dans la fonction "main" :

toto.c:5: attention : suggest explicit braces to avoid ambiguous 'else'

Comprendre les messages d'erreur des
compilateurs

Savoir programmer en assembleur

Savoir écrire un compilateur

Savoir quelles optimisations les compilateurs font
automatiquement

Planning

11 cours, 1 partiel, 12 TD sur machine

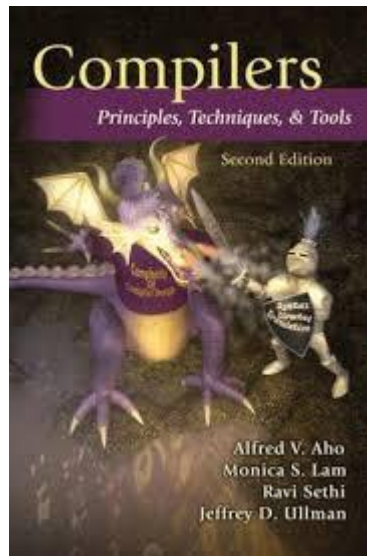
1 partiel 30 %

1 projet logiciel 40 %

1 examen final 30 %

$$nf = \max((3np + 4npl + 3nef) / 10, (2npl + 3nef) / 5)$$

Bibliographie



Aho, Sethi, Ullman, 1986/2007. *Compilateurs. Principes, techniques et outils*, Pearson Education France

Levine, Mason, Doug, 1990. *Lex & Yacc*, O'Reilly.

Ce support de cours est inspiré de Aho *et al.* (1986)

Sommaire

Présentation du cours

Projet

Moteur d'exécution

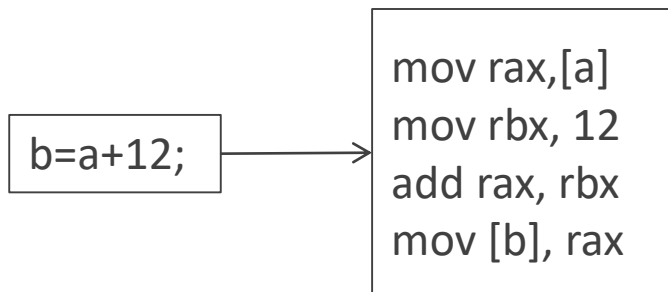
Activations des fonctions

Adresses

Table des symboles

Le projet de compilation

TPC $\longrightarrow$ assembleur nasm



Objectif

Écrire un compilateur en C

Langage source

TPC, presque un sous-ensemble du C

Langage cible

Assembleur nasm 64 bits

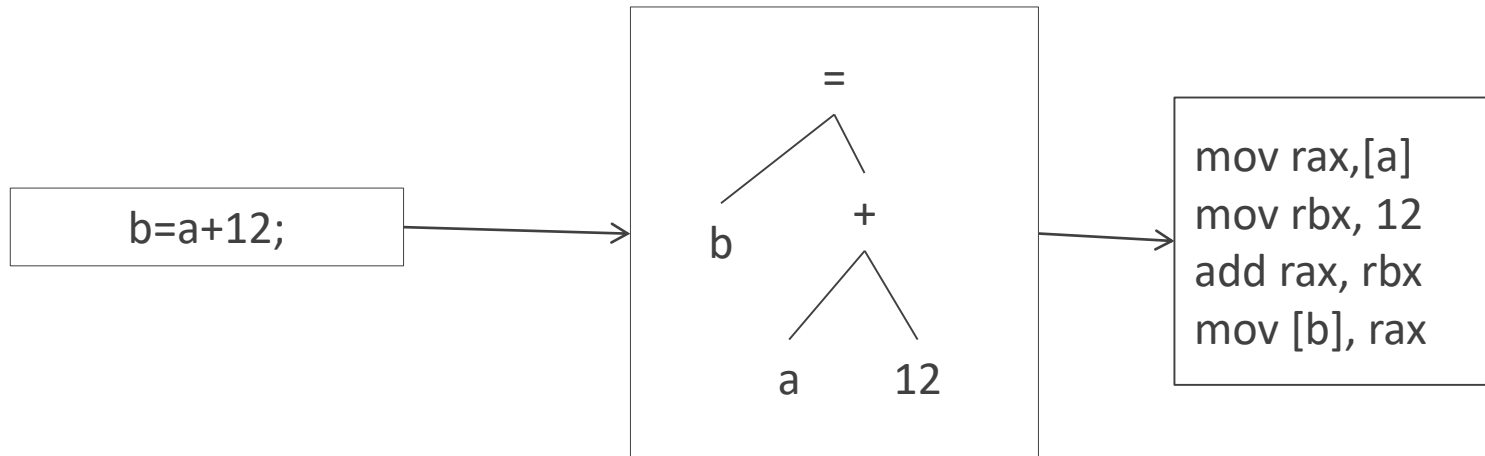
Analyse syntaxique

Perfectionner votre projet d'analyse syntaxique

Date limite de rendu

23 mai

Le projet de compilation en deux passes



expression → arbre abstrait → programme en nasm 64 bits

Compilation en une passe

Traduire le programme source directement dans
un code intermédiaire ou final
Analyse syntaxique et traduction en une seule
passe (possible en Bison)

Retour sur le projet d'analyse syntaxique

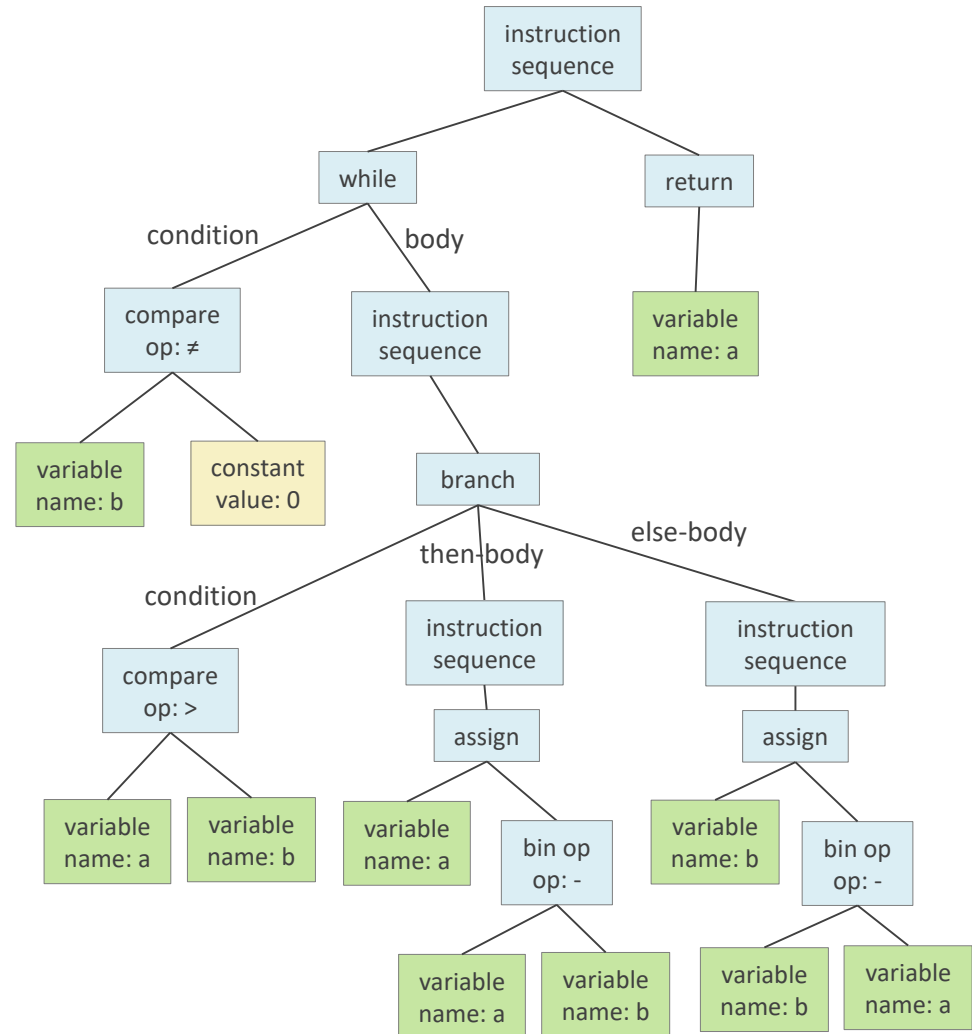
Bonne qualité en général

À perfectionner pour le projet de compilation

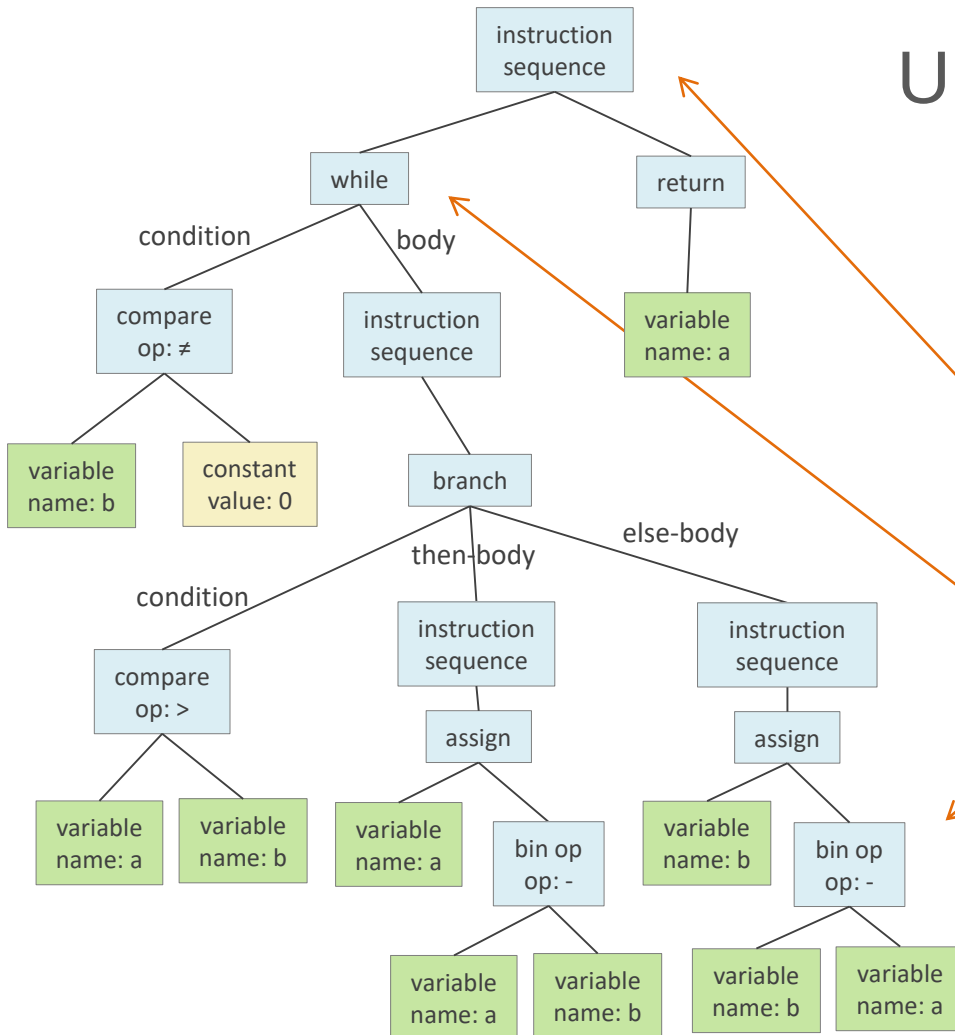
Arbre abstrait

Un arbre abstrait satisfaisant

```
while b != 0
  if a > b
    a := a - b
  else
    b := b - a
return a
```



Un arbre abstrait satisfaisant



Liste

Un seul nœud pour la liste, les éléments comme fils

Structure de contrôle

Un nœud interne, chaque partie comme fils

Opérateur

Un nœud interne, les opérandes comme fils

Attributs de nœuds de l'arbre abstrait

Informations nécessaires pour la compilation

Exemple : valeurs des constantes

... pour traduire en code cible

Valeurs des constantes

Identificateurs (seront remplacés par des adresses)

... pour l'analyse sémantique

Vérifier les contraintes supplémentaires par rapport à l'analyse syntaxique

- on doit déclarer les identificateurs avant de les utiliser
- les opérateurs doivent être compatibles avec le type des opérandes

- ...

Sommaire

Présentation du cours

Projet

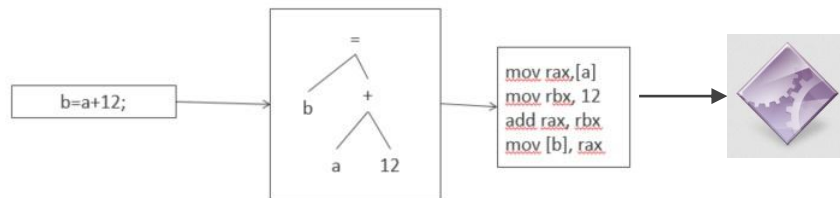
Moteur d'exécution

Activations des fonctions

Adresses

Table des symboles

Moteur d'exécution (*runtime system*)

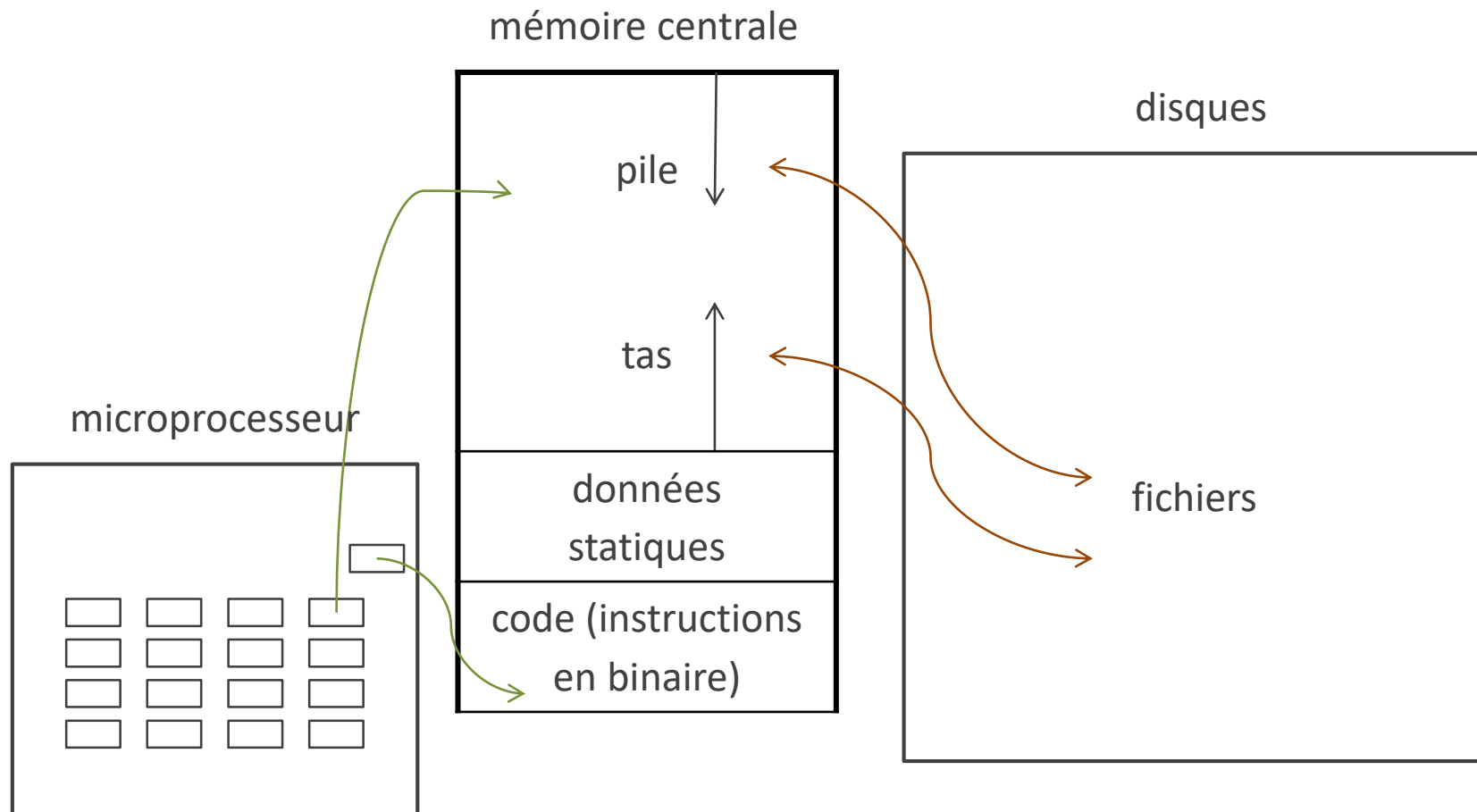


Ce que fait le système d'exploitation pour exécuter un programme en binaire

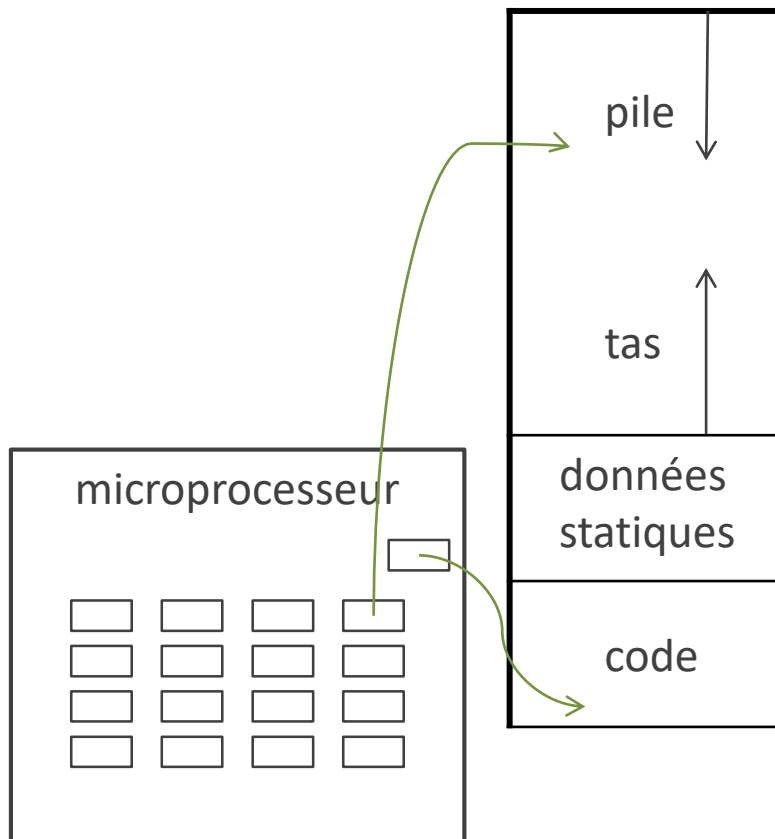
Il faut savoir ce que fait le moteur d'exécution pour écrire dans un langage de bas niveau

- Il charge le programme
- Il passe d'une instruction à une autre
- Interface avec le système d'exploitation

Organisation de la mémoire



Organisation de la mémoire



Charger le programme

Copier en mémoire - l'exécutable
- les arguments en ligne de commande

Initialiser les registres : - compteur ordinal
- registre de pile

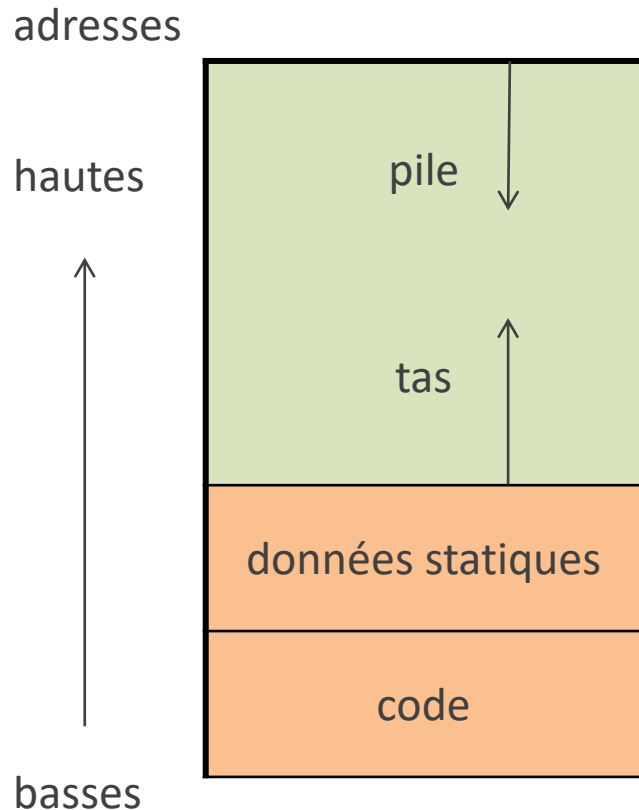
Passer d'une instruction à une autre

Incrémenter le compteur ordinal

Interface avec le système d'exploitation

Lire, écrire dans des fichiers

Organisation de la mémoire



Partie dynamique

Les adresses des données sont valables seulement pendant une partie du programme

Pile d'exécution

Tas : en C, la mémoire allouée dynamiquement

Les adresses dépendent de chaque exécution

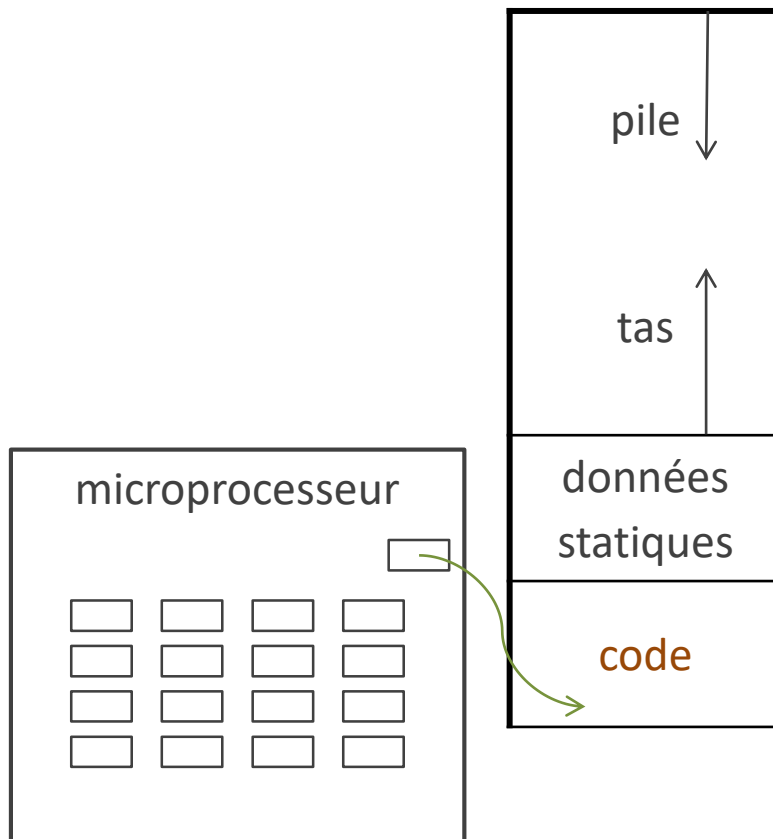
Partie statique

Données statiques : dont les adresses ne changent pas pendant la durée du programme

Le code ne change pas pendant l'exécution

Les adresses sont en dur dans le code et connues à la compilation

Statique ou dynamique



Le code du programme

Statique

Connu à la compilation

Le compteur ordinal

Passe d'une instruction à une autre

Dynamique

Les valeurs dépendent de chaque exécution

Fichier de démarrage pour le langage C

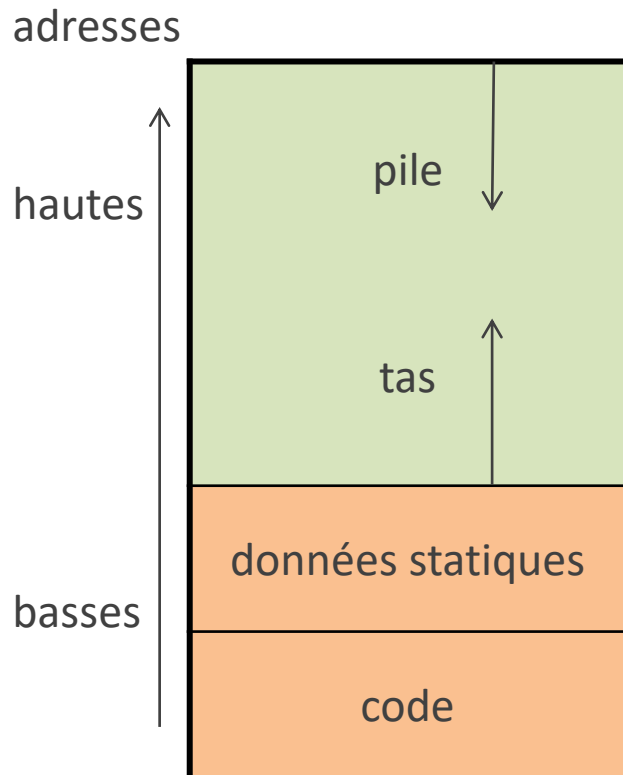
```
gcc -o my_prog my_prog.o utils.o
```

En plus des modules indiqués, l'éditeur de liens lie un fichier de démarrage (*start file*) `crt0.o` (*C runtime object*) qui appelle `main()`

Le moteur d'exécution initialise le compteur ordinal à l'adresse d'une instruction du fichier de démarrage

Dans le projet, le lancement de l'exécution sera un peu différent

La mémoire centrale



Variables

Dans la pile ou dans les données statiques

Constantes

Constantes littérales du C (**#define**) :
dans le code

Variables constantes du C++ (**const**) :
dans les données statiques ou la pile

Résultats intermédiaires des calculs

Dans la pile

Sommaire

Présentation du cours

Projet

Moteur d'exécution

Activations des fonctions

Adresses

Table des symboles

Activations

Fonction

A un nom (identificateur) et un corps (instruction)

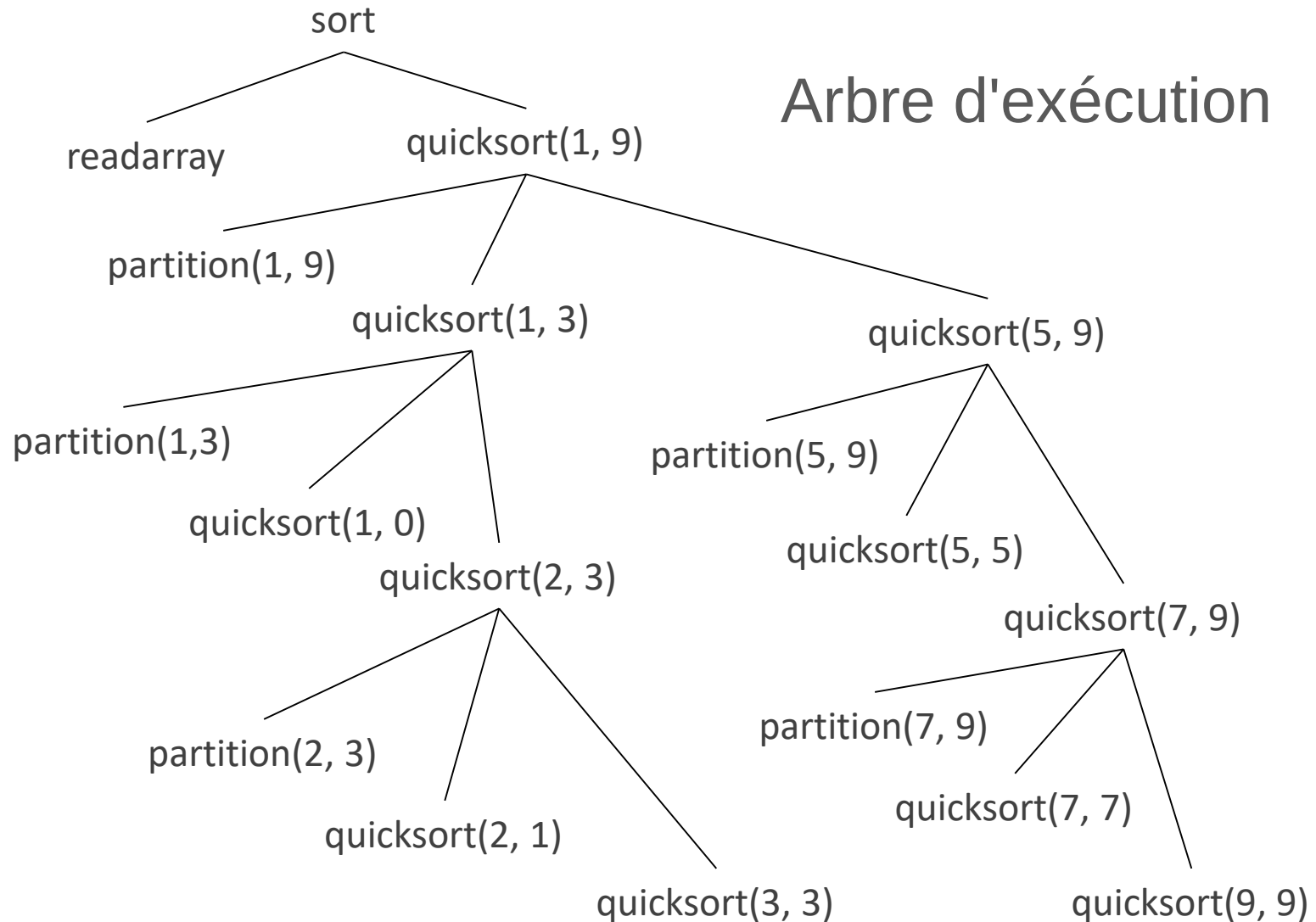
Activation d'une fonction

Période qui va de l'appel d'une fonction (avec éventuellement des paramètres) à son retour

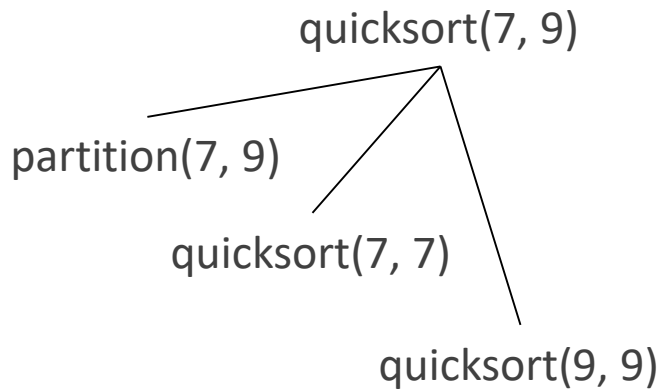
Fonction récursive

Fonction dont une activation peut commencer alors qu'une autre est en cours

Arbre d'exécution



Pile d'exécution



Arbre d'exécution

Les nœuds sont les activations des fonctions avec leurs paramètres

Chaque fils d'un nœud correspond à un appel

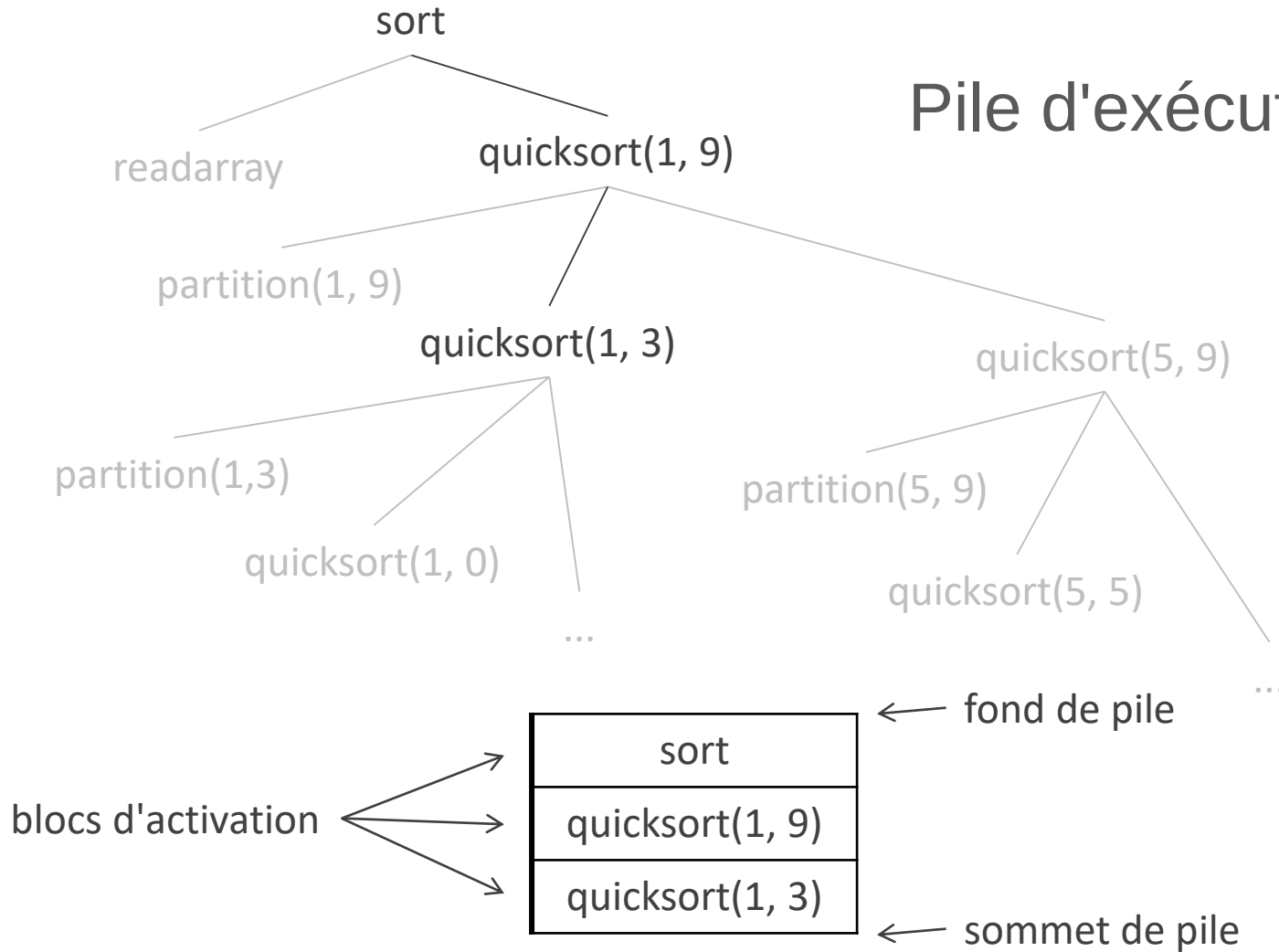
Pile d'exécution

La pile des activations en cours à un instant donné

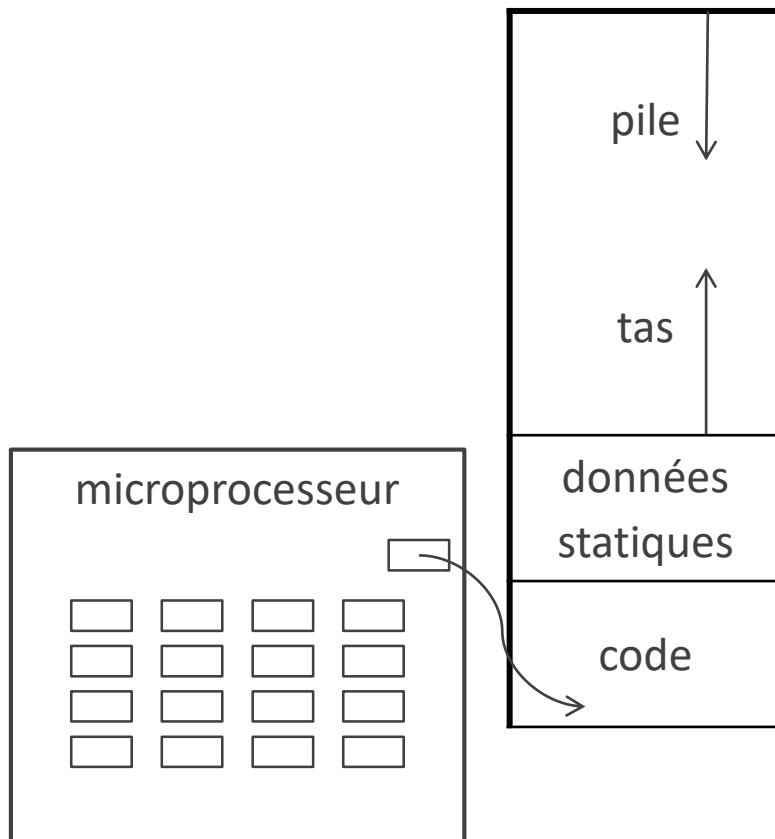
L'instruction en cours d'exécution fait partie de la fonction dont le **bloc d'activation** ou **enregistrement d'activation** est en sommet de pile

Les autres activations en cours sont suspendues

Pile d'exécution



Appel et retour de fonction



Appel de fonction

Le moteur d'exécution

- sauvegarde l'adresse de retour dans la pile
- saute à l'adresse de la fonction appelée

Retour d'une fonction

Le moteur d'exécution

- dépile l'adresse de retour
- saute à l'adresse de retour

Différences entre langages

Langages

C, Java, C++, Ada, Python, Perl

Pascal, Lisp, Fortran, Algol, APL, Snobol...

Un langage sans fonctions récursives : Fortran 77

L'organisation d'un compilateur dépend des
caractéristiques du langage source

Sommaire

Présentation du cours

Projet

Moteur d'exécution

Activations des fonctions

Adresses

Table des symboles

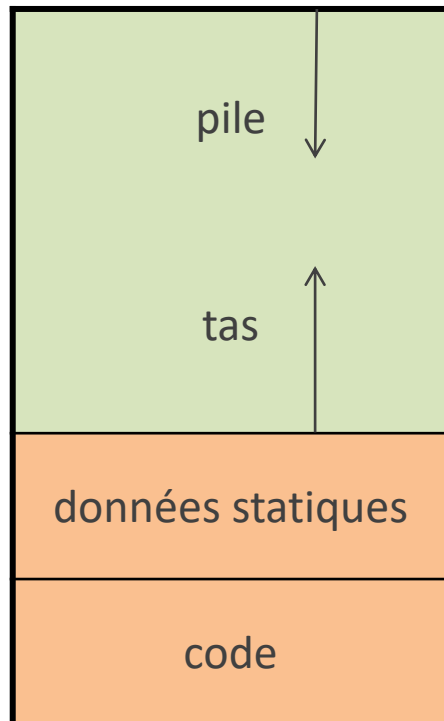
Organisation de la mémoire

adresses

hautes



basses



Allocation automatique

À l'entrée des blocs

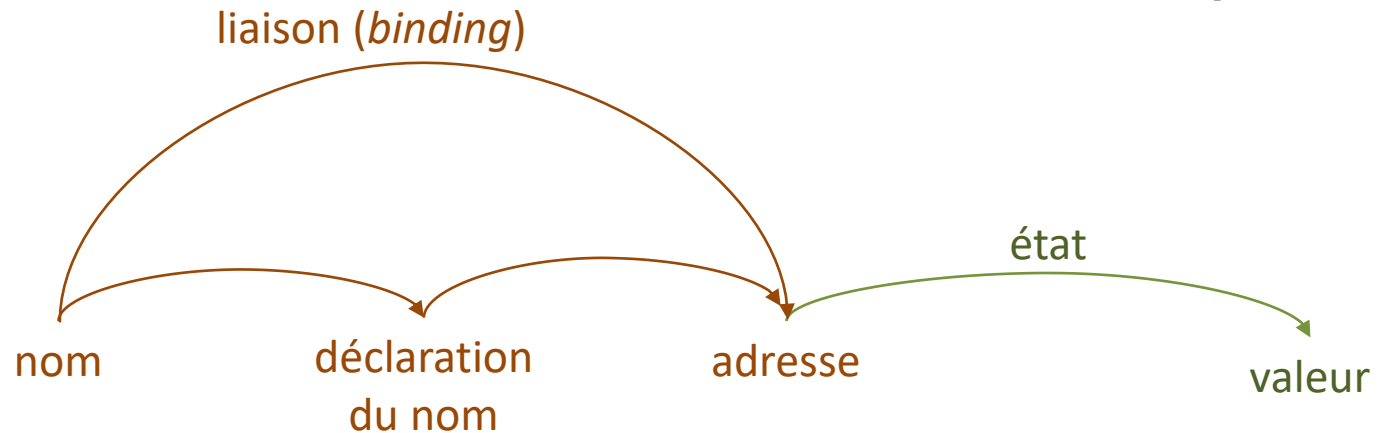
Les adresses dans la pile sont des adresses relatives (*offsets*) par rapport à une certaine position dans la pile (base)

Allocation dynamique

Sur demande explicite dans le source

Allocation statique

Allocation statique



L'adresse est connue à la compilation

Si l'environnement associe une occurrence du nom *id* à l'adresse *a*, on dit que *id* est lié à *a*

La valeur stockée à une adresse dépend de chaque exécution

Une adresse peut contenir plusieurs valeurs successivement

Allocation dynamique

En langage C, indépendante de la compilation

Utilise le tas

**Libération de la mémoire allouée
dynamiquement**

Différences entre langages

- manuelle : C
- automatique (ramasse-miettes, *garbage collector*) : Java, Python, Perl
- manuelle ou automatique : Ada

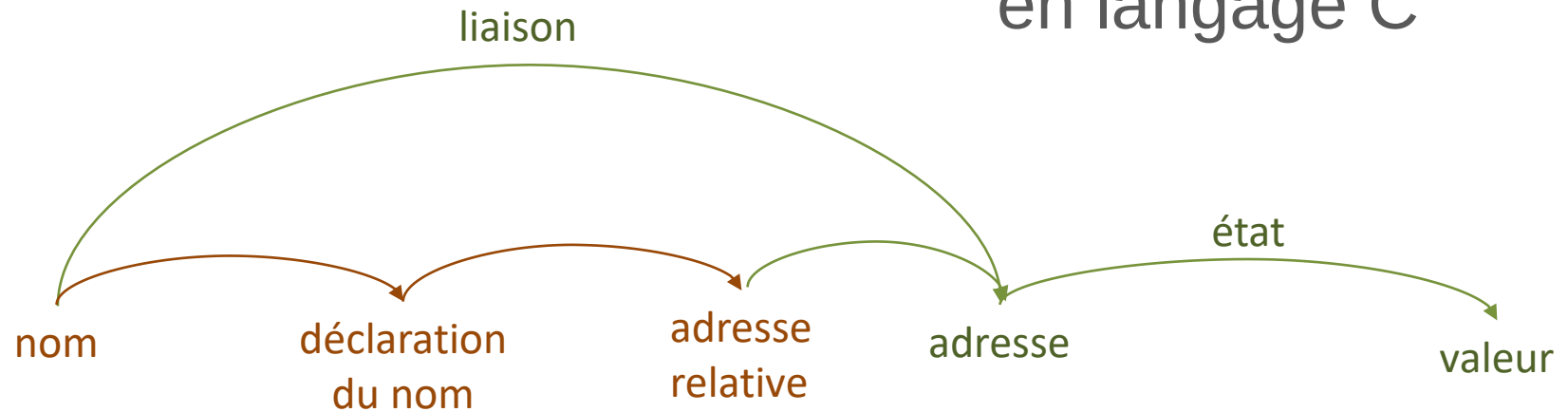


Ada Lovelace par Margaret Sarah Carpenter

<http://www.gac.culture.gov.uk/work.aspx?obj=28401>

Public Domain, <https://commons.wikimedia.org/w/index.php?curid=28993870>

Allocation automatique en langage C



L'adresse relative est connue à la compilation

Sommaire

Présentation du cours

Projet

Moteur d'exécution

Activations des fonctions

Adresses

Table des symboles

Table des symboles

tab	
a	
x	
input_array	
i	
n	

Mémoire les informations sur les identificateurs contenus dans le programme

Objectifs : détecter des erreurs, mettre dans le programme cible les adresses des variables et des fonctions

Entrées d'une table des symboles

Correspondent à des déclarations

Si un même nom est déclaré plusieurs fois, il a plusieurs entrées dans la même table ou dans plusieurs tables des symboles

Table des symboles

tab	
a	
x	
input_array	
i	
n	

Le contenu d'une table des symboles évolue pendant la compilation

Quand on entre dans un bloc, on ajoute des entrées pour les nouvelles variables locales

À la fin de la compilation, les tables des symboles disparaissent

Opérations sur une table des symboles

- lire l'entrée correspondant à un identificateur
- insérer une nouvelle entrée
- initialiser la table des symboles

Insérer une nouvelle entrée pour une déclaration

tab	
a	
x	
input_array	
i	
n	

Indiquer ce qu'on sait sur la variable déclarée

Décider où le programme réservera de la mémoire
pour la variable à l'exécution

Type

Les variables d'un même type occupent la même
taille mémoire

Adresse relative

Indiquer dans une variable `dep1ct` l'adresse du
prochain emplacement mémoire disponible

Une table des symboles ou plusieurs ?

tab	
a	
x	
input_array	

i	
n	

Si une fonction utilise une variable locale déclarée dans une autre, c'est une erreur sémantique

Comment détecter l'erreur ?

Solution 1 : plusieurs tables des symboles

Une pour chaque fonction
plus une pour les variables globales

Solution 2 : une seule table des symboles

Compiler chaque fonction séparément

Quand on a fini de compiler une fonction, retirer de la table les entrées de ses variables locales

Table des symboles

tab	
a	
x	
input_array	
i	
n	

Implémentation avec table de hachage ouverte

N seaux (*buckets*)

Une liste chaînée dans chaque seau

Quand on trouve un identificateur, on le met en tête de la liste pour le cas où on le retrouverait bientôt

Exemple de fonction de hachage

Identificateur : $c_1c_2...c_n$

$$h_0 = 0$$

$$h_i = k h_{i-1} + c_i$$

$$H = (h_n \bmod 2^{30}) \bmod N$$

$$k = 613$$

$$N = 1008$$

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR